CSEN 122L: Project (Spring 2026)

Design a Structural Model of a Pipelined CPU

Description: The project is to design a structural model of a pipelined CPU with 12 instructions using Verilog HDL. In this project, you will design a 32-bit pipelined CPU for the given SCU Instruction Set Architecture (SCU ISA). The SCU ISA is described below.

- Register file size: 64 registers, each register has 32 bits. The 64 register names are referred to as `x0`, `x1`, `x2`, ..., `x63`.
- PC: 32 bits
- Instruction format: Each instruction is 32-bit wide, and consists of five fields: `opcode` (operation code), `rd` (destination register), `rs` (first source register), `rt` (second source register), and `y` (immediate value). The format is as follows.

opcode (4 bits)	rd (6 bits)	rs (6 bits)	rt (6 bits)	y: immediate value (10 bits)

The 12 instructions are defined in the following table (**X** = don't care / unused)

Instruction	Syntax	Opcode	rd	rs	rt	y	Function
No operation	`NOP`	0000	X	X	X	X	No operation
Add	`ADD rd,rs,rt`	0100	rd	rs	rt	X	xrd <- xrs + xrt
Subtract	`SUB rd,rs,rt`	0111	rd	rs	rt	X	xrd <- xrs - xrt
Negate	`NEG rd,rs`	0110	rd	rs	X	X	xrd <- -xrs
Increment	`INC rd,rs,y`	0101	rd	rs	y		xrd <- xrs + y (16 bits) (y sign-extended)
Save PC	`SVPC rd,y`	1111	rd	y			xrd <- PC + y (22 bits) (y sign-extended)
Load	`LD rd,rs`	1110	rd	rs	X	X	xrd <- M[xrs]
Store	`ST rt,rs`	0011	X	rs	rt	X	M[xrs] <- xrt
Jump memory	`JM rs`	1000	X	rs	X	X	PC <- M[xrs]
Branch if zero	`BRZ rs`	1001	X	rs	X	X	PC <- xrs, if Z=1
Branch if negative	`BRN rs`	1011	X	rs	X	X	PC <- xrs, if N=1
ADDV	`ADDV rd, rs, rt`	0001	rd	rs	rt	X	a = a + b, see *

*M[xrs + i] = M[xrs + i] + M[xrt + i] for i = 0, …, x**rd**. x**rs** has the base address of vector a. x**rt** has the base address of vector b. x**rd** has value n-1.

SCU ISA CPU Characteristics

(1) Word addressing architecture: Unlike RISC-V, SCU ISA uses word addressing, not byte addressing. In other words, in the word addressing architecture, each individual address indicates a different word (32 bits). In byte addressing, on the other hand, each individual address indicates a different byte (8 bits).

Suppose that we have the following code with SCU ISA:

```
1000: SVPC x5,1          // x5 = PC + 1
1001: ADD  x6,x7,x8      // x6 = x7 + x8
1002: SUB  x9,x10,x11    // x9 = x10 - x11
```

Assuming the current PC is 1000 executing the SVPC instruction "`SVPC x5,1`", then the x5 register will be updated to hold a value of 1001 that indicates the address of `ADD` ("`ADD x6,x7,x8`") instruction.

(2) Branch instructions use the zero/negative flag from the previous instruction executed immediately before. Our branch instructions, "Branch if Zero (`BRZ`)" and "Branch if Negative (`BRN`)", have only one operand that indicates the branch target address. For branch condition, the branch instruction needs to look at the Zero (`Z`) or Negative (`N`) flag from the instruction executed immediately before the branch instruction. The Zero flag is 1 if the previous ALU result was 0. The Negative flag is 1 if the previous ALU result was a negative value.

Suppose that we have the following code:

```
1000: SVPC x5,1         // x5 <- PC + 1
1001: ADD  x6,x7,x8     // x6 <- x7 + x8
1002: SUB  x9,x10,x11   // x9 <- x10 - x11
1003: BRN  x5           // PC <- x5, if x9 < 0
```

The branch instruction BRN will set the PC to indicate the ADD ("`ADD x6,x7,x8`") instruction if the result of the SUB instruction ("`SUB x9,x10,x11`") is a negative value.

Also, refer to the ALU block diagram and truth table in the last page of this document.

Assembly programming (the first assignment)

Write an assembly program using the 11 base instructions of the SCU ISA (`NOP`, `ADD`, `SUB`, `NEG`, `INC`, `SVPC`, `LD`, `ST`, `JM`, `BRZ`, `BRN`) to do **vector addition**, as described below.

You may use a text editor or a word processor to write your assembly code. Finally, convert your implementation to a PDF and submit it **on Camino**.

Vector add program

Write an assembly program that does vector addition, such as

$$a_i = a_i + b_i \text{ , where } i = 0, \ldots, n-1$$

This program processes one-dimensional input arrays **a** and **b** consisting of **n integer elements**, with each element occupying **4 bytes** of memory.

You will use the instructions in the SCU ISA to write two versions of assembly program of the benchmark below where two vectors **a** and **b** are added and the results are saved back to **a**. The first version does not use the ADDV instruction, and the second one uses the ADDV instruction. This program will also be used to test your CPU.

When you write the assembly code, you can assume the **problem size *n*** (the number of vector elements to add) and the **base address of array `a`** (`&a[0]`) and **b** (`&b[0]`) are in some registers. Specify your assumptions in your submission and write some comments about your logic.

Zero register: If you want to use a specific register for hard-wired 0 value, you can do that, but specify that in your submission and also in your project report if you implemented your CPU to do that. You can also initialize a specific register with a value of 0, using a simple sub instruction, e.g.

```
sub x0, x0, x0
```

The last instruction ADDV is optional. However, two extra points will be added to both CSEN 122 and 122L if the ADDV instruction is implemented correctly in your CPU.

Assignments and Submission:

Note: **One submission per team**.

1. The two versions of assembly code for the vector addition program to Dr. Shang on the Saturday of Week 3. Please upload your solution to Camino CSEN 122 lecture.

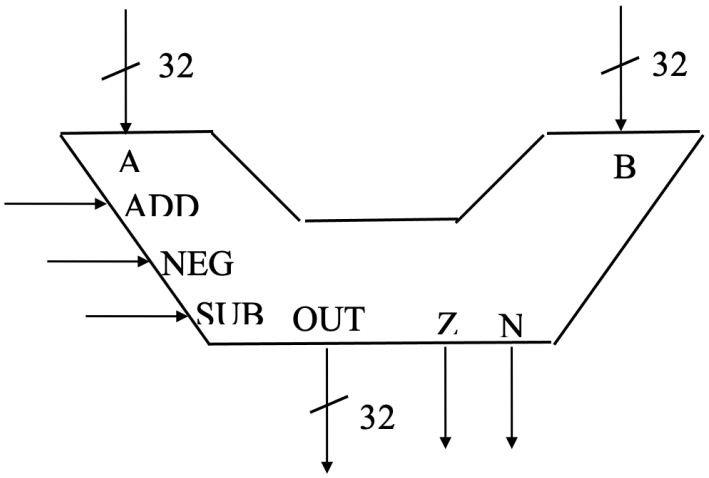
2. The datapath and control (including the truth table) for the SCU ISA that supports the 11 SCU ISA instructions on the Friday of Week 8 to Dr. Shang. Use the ALU attached in the last page. **Submit** your datapath and control truth table to Camino CSEN 122 lecture.

3. Code & Demo: Please **demo** your working program to the TA **(during the Lab sections or TA office hours)** and **submit** your working code to Camino. During the demo, the TA will provide you *n* random numbers for each input array of the vector addition program, and you will show the TA the result after running your program on your pipeline. You will also be asked to perform random but related tasks (for instance, change the address of data and demo the modified program), and be prepared to demo/answer related project questions. The purpose of these questions is to make sure that you understand the project thoroughly.

4. Report: **Submit** a written final report to the lab Camino in a PDF format including the following:
   - ➢ Abstract (short description or outline of the project)
   - ➢ Detailed description of the CPU design including the datapath and the truth table of your control
   - ➢ Test benchmarks/waveforms verifying the functions
   - ➢ The **final** vector addition program assembly code for the benchmark
   - ➢ Performance analysis: estimate CPI, express the instruction count, total number of cycles in terms of the problem size *n*, and verify your estimate with simulation/waveform. When you analyze the cycle time, you can use the following delay data: delay of memory (I and D memory): 2 ns., delay of register file: 1.5 ns., delay of ALU (adders): 2 ns. Ignore the delays of all other components. Use the ALU attached in the last page.

Appendix (ALU block diagram and ALU truth table):

ALU Block Diagram



ALU Truth Table

ADD	NEG	SUB	OUT	Operation
0	0	0	A + B	Add
0	1	0	Don't Care	No Operation
1	0	1	A - B	Subtract
1	1	0	-A	2's Complement
1	1	1	A	Pass A

Z=1 if and only if OUT=0
N=1 if and only if OUT is negative.